Static Covariates: Predicting density and assessing factors driving marbled murrelet distribution in Puget Sound using hierarchical distance sampling

Code by S.J. Gillman

Description & Sources

There are 1545 grids within the study area. Additionally, there are 8,095 PSU-segment tracklines that were surveyed between 2012 and 2024. For each grid cell and PSU-segment, I will be calculating or assigning the following:

1. **Depth (meters)** *as a weighted average*

- This is calculated from the `SS_Bathymetry.tiff` which is the Bathymetric Base Map for the Salish Sea Bioregion by Aquila Flower, 2021.

2. **Bathymetric Position Index**

- Bathymetric Position Index (BPI) is the difference between the value of a focal cell and the mean of the surrounding cells contained within an annulus shaped window (Lundblad et al., 2006).
- To calculate the *Bathymetric Position Index*. I use the `MultiscaleDTM` package which corresponds to the paper MultiscaleDTM: An open-source R package for multiscale geomorphometric analysis

3. **DClass Shore**

- `Dclass_cut.gpkg` file to calculate the distance from each grid's centriod to the nearest shoreline. I will constrain the centroids of the grids to force them to be in the water.

4. **The Pacific Marine and Estuarine Fish Habitat Partnership**

- Coastal and Marine Ecological Classification Standard (CMECS) out to -200 Substrate Component (SC). Uses the files `pmep_substrat_habitat.gpkg`

Required Packages

```
library(sf)
library(tidyverse)
library(MultiscaleDTM)
library(viridis)
```

Required Files Previously Made

- `puget_poly.gpkg`
- `grid2km_hex.gpkg`
- `all_tracks_2026_updated.gpkg`
- `Dclass_cut.gpkg`
- `reduced_area.gpkg`

```r
ss_bath <- terra::rast("../GIS_COVS/SS_Bathymetry/SS_Bathymetry.tif")
ss_bath

puget <- st_read("../GIS_COVS/GPKG/puget_poly.gpkg")
st_crs(puget)

hex_grids <- st_read("GIS_COVS/grid2km_hex.gpkg")
st_crs(hex_grids)

tracklines <- st_read("../GIS_COVS/GPKG/all_tracklines_2026_final.gpkg")
st_crs(tracklines)

track_buffer <- tracklines %>%
  mutate(numid = row_number()) %>%
  st_buffer(dist = 215)

shoreline <- st_read("../GIS_COVS/GPKG/Dclass_cut.gpkg")
st_crs(shoreline)
studyarea <- st_read("../GIS_COVS/GPKG/reduced_area.gpkg")
st_crs(studyarea)
```

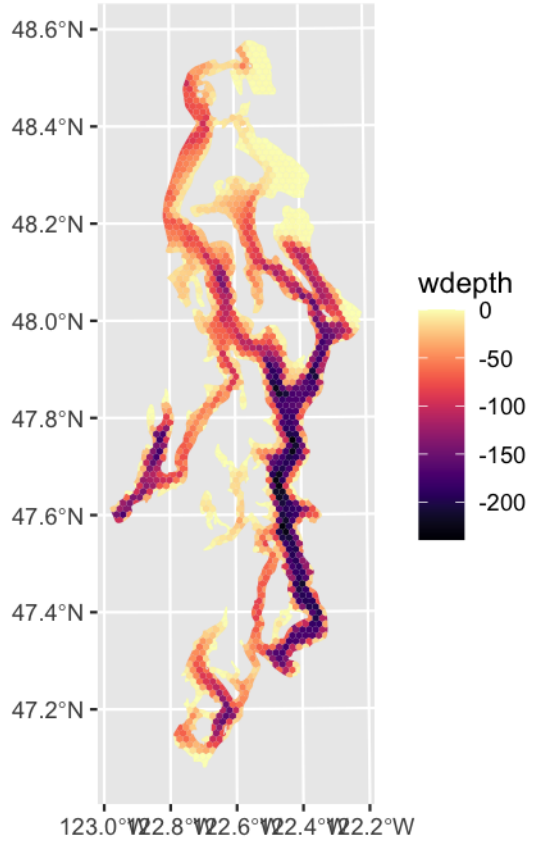
Bathymetry

Depth (m)

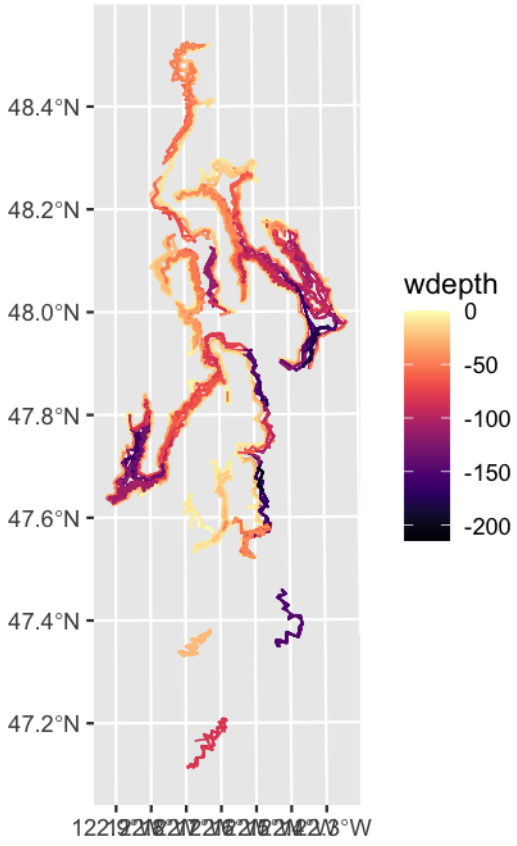
Cropping and Masking the bathymetry. They both have the same projection so there is no need to transform.

```r
# crop raster to remove land values with shoreline
cropped_raster <- crop(ss_bath, puget)
#mask
masked_raster <- terra::mask(cropped_raster, puget)


#ggplot()+
#  tidyterra::geom_spatraster(data =masked_raster)+
#  scale_fill_viridis_c(direction=-1)

depth_values_hex <- terra::extract(masked_raster, hex_grids,
                                   fun = "mean",weights= T,ID=T,na.rm=T)
head(depth_values_hex)



hex_depth <- hex_grids %>%
  full_join(depth_values_hex %>%
              mutate(SS_Bathymetry = ifelse(SS_Bathymetry > 0, 0,
                                            SS_Bathymetry)) %>%
              rename(numid= ID, wdepth = SS_Bathymetry), by = "numid")

any(is.na(hex_depth$wdepth))

ggplot()+
```

```
  geom_sf(data = hex_depth, aes(fill = wdepth), color =NA)+
  scale_fill_viridis(option="A")
```

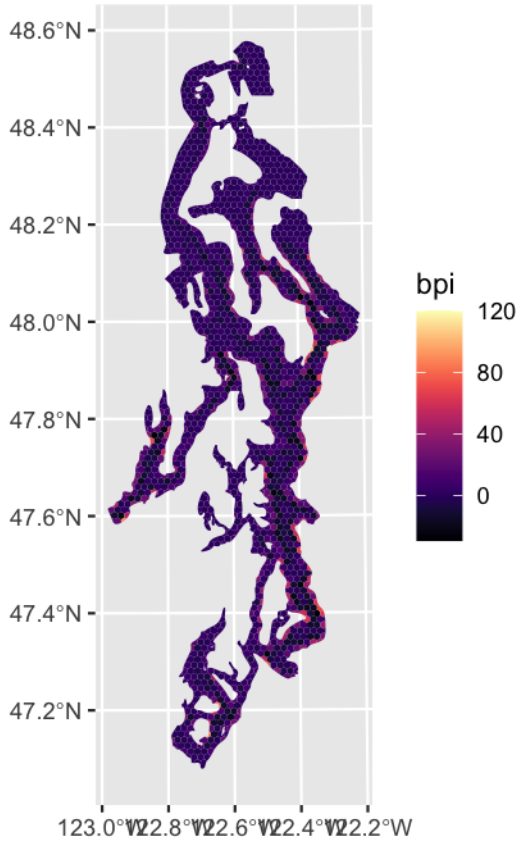



```
depth_values_track <- terra::extract(masked_raster, track_buffer,
                                     fun = "mean",weights= T,ID=T,na.rm=T)
head(depth_values_track)



track_depth <- track_buffer %>%
  full_join(depth_values_track %>%
              mutate(SS_Bathymetry = ifelse(SS_Bathymetry > 0, 0,
                                            SS_Bathymetry)) %>%
              rename(numid= ID, wdepth = SS_Bathymetry), by = "numid")

any(is.na(track_depth$wdepth))

ggplot()+
  geom_sf(data = track_depth, aes(fill = wdepth), color=NA)+
  scale_fill_viridis(option="A")
```

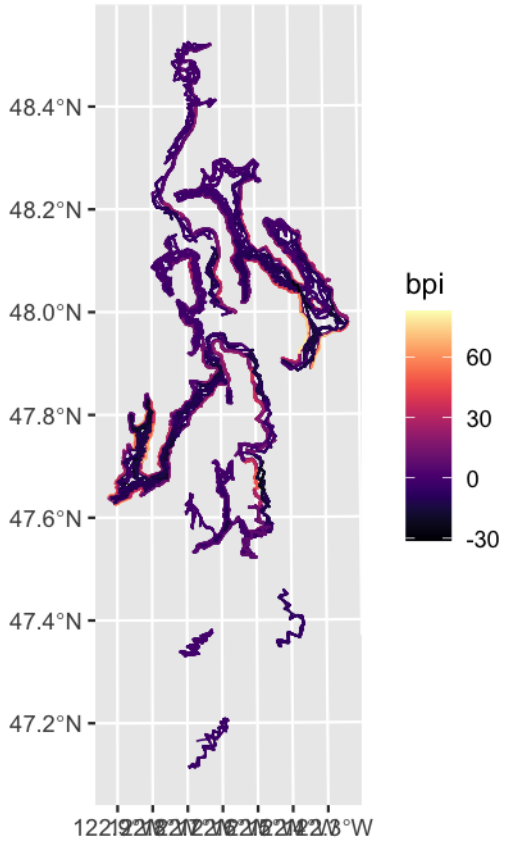


BPI

```
bpi<- BPI(masked_raster, w = annulus_window(radius = c(540,900),
                                            unit = "map",
                                            resolution = 90),na.rm = TRUE)
plot(bpi)

bpi_ext <- terra::extract(bpi, hex_depth,fun = "mean",
                          weights= T,ID=T,na.rm=T)


hex_bpi_d <- hex_depth %>%
  full_join(bpi_ext %>%
              rename(numid= ID), by = "numid")

ggplot()+
  geom_sf(data = hex_bpi_d, aes(fill = bpi), color=NA)+
  scale_fill_viridis(option="A")

#hist(hex_bpi_d$bpi)
```



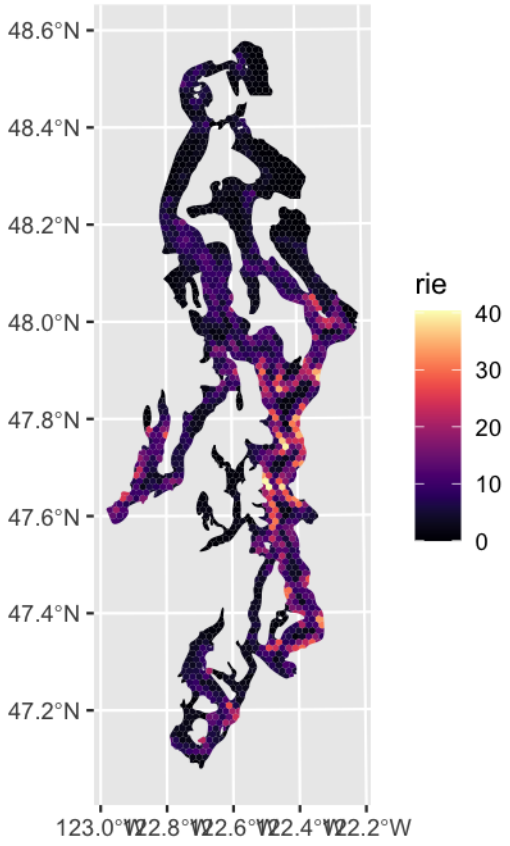

```
bpi_ext_track <- terra::extract(bpi, track_depth,
                                fun = "mean",weights= T,ID=T,na.rm=T)


track_bpi_d <- track_depth %>%
  full_join(bpi_ext_track %>%
              rename(numid= ID), by = "numid")

ggplot()+
  geom_sf(data = track_bpi_d, aes(fill = bpi), color=NA)+
  scale_fill_viridis(option="A")

#hist(track_bpi_d$bpi)
```



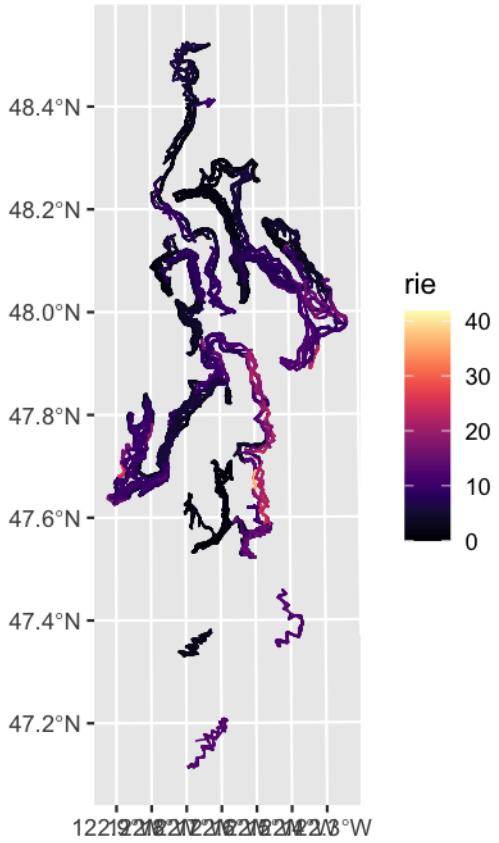
RIE

Calculates the Roughness Index-Elevation which quantifies the standard deviation of residual topography (Cavalli et al., 2008). This measure is conceptually similar to AdjSD but rather than fitting a plane and extracting residuals for the entire focal window, residual topography is calculated as the focal pixel minus the focal mean. Then the local standard deviation is calculated from this residual topography using a focal filter.

```
rie<- RIE(masked_raster, w=c(15,15), na.rm = TRUE)

rie_ext<- terra::extract(rie, hex_depth,fun = "mean",
                         weights= T,ID=T,na.rm=T)

hex_rie_bpid <- hex_bpi_d %>%
  full_join(rie_ext %>%
              rename(numid= ID), by = "numid") %>%
  mutate(rie = ifelse(is.na(rie),0, rie))


# Identify rows with NA values

ggplot()+
  geom_sf(data = hex_rie_bpid, aes(fill = rie), color=NA)+
  scale_fill_viridis(option="A")
```




```
rie_ext_track<- terra::extract(rie, track_depth,fun = "mean",
                               weights= T,ID=T,na.rm=T)

track_rie_bpid <- track_bpi_d %>%
  full_join(rie_ext_track %>%
              rename(numid= ID), by = "numid") %>%
  mutate(rie = ifelse(is.na(rie), 0, rie))

ggplot()+
  geom_sf(data = track_rie_bpid, aes(fill = rie), color=NA)+
  scale_fill_viridis(option="A")
```



Distance to shore

This will use the `Dclass_cut.gpkg` file to calculate the distance from each grid's centriod to the nearest shoreline. I will constrain the centroids of the grids to force them to be in the water.

```
#st_crs(shoreline)
any(!st_is_valid(shoreline))
```

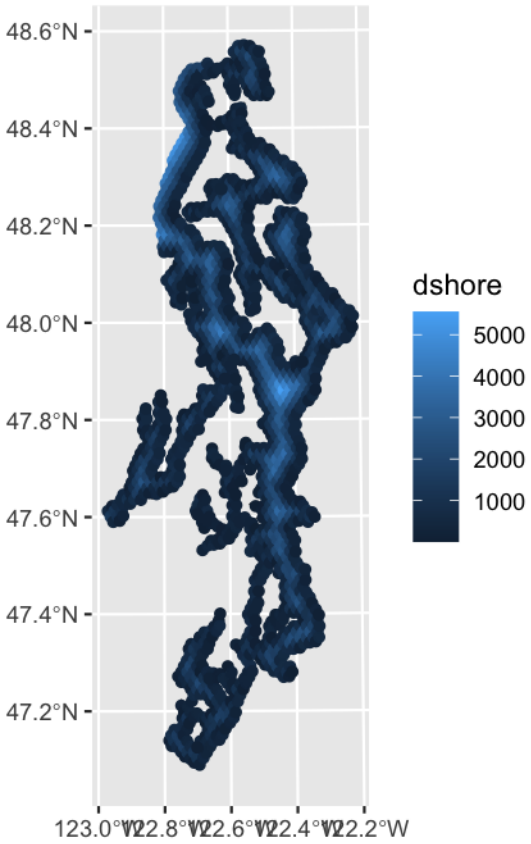
```
## [1] FALSE
```

```
# **Compute centroids for all polygons**
centroids_in_water <- st_centroid(hex_grids)
inside_points <- st_point_on_surface(hex_grids)

# **Check which centroids are inside the polygons (convert to logical vector)**
centroid_inside <- apply(st_contains(hex_grids, centroids_in_water,
                                     sparse = FALSE), 1, any)

# **Weight factor for interpolation (0 = fully inside, 1 = exact centroid)**
w <- 0.4  # Adjust this value to control the shift

# **Interpolate coordinates to "push" points toward the center**
coords_inside <- st_coordinates(inside_points)
coords_centroid <- st_coordinates(centroids_in_water)
```

```
adjusted_coords <- coords_inside * (1 - w) + coords_centroid * w

# **Convert adjusted coordinates back into an `sfc` object**
adjusted_points <- st_sfc(lapply(seq_len(nrow(adjusted_coords)), function(i) {
  st_point(adjusted_coords[i, ])
}), crs = st_crs(hex_grids))

# **Properly update only the centroids that are outside**
centroids_in_water$geom[!centroid_inside] <- adjusted_points[!centroid_inside]

any(!apply(st_contains(hex_grids, centroids_in_water,
                       sparse = FALSE), 1, any))
```

```
## [1] TRUE
```

```
centroids_in_water$dshore <-  as.numeric(st_distance(
  st_union(st_geometry(shoreline)),
  centroids_in_water))


ggplot()+
  geom_sf(data = puget)+
  geom_sf(data=centroids_in_water, aes(color = dshore))
```



Benthic data

The Pacific Marine and Estuarine Fish Habitat Partnership Coastal and Marine Ecological Classification Standard (CMECS) out to -200 Substrate Component (SC). GPKG file created in `QGIS` and the original files are provided.

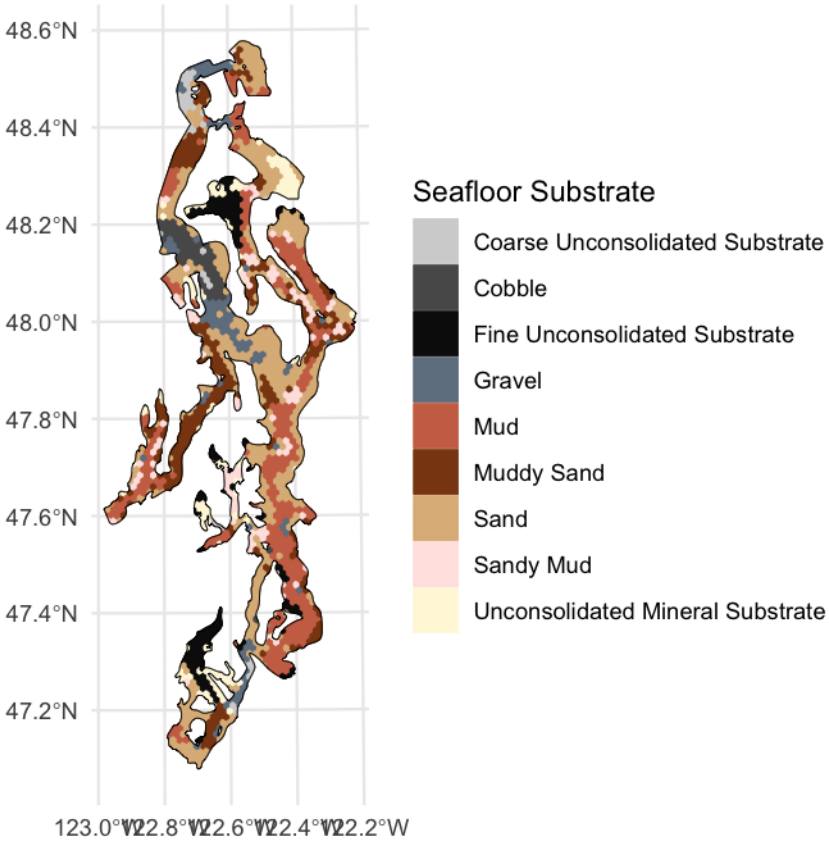
```
substrate <- st_read("../GIS_COVS/GPKG/pmep_substrat_habitat.gpkg")
```

Instead of re categorizing before, will see what overlaps with grids and then change

```
hex_substrate <- hex_grids %>%
  st_intersection(substrate %>%
                    dplyr::select(CMECS_SC_Name)) %>%
  mutate(sub_area = as.numeric(st_area(geom))) %>%
  st_drop_geometry() %>%
  dplyr::select(c(ID, numid,sub_area,CMECS_SC_Name))%>%
  group_by(ID, numid,CMECS_SC_Name) %>%
  dplyr::summarize(sub_area= sum(sub_area)) %>%
  ungroup() %>%
  distinct()

hex_substrate2 <- hex_grids %>%
  st_intersection(substrate %>%
                    dplyr::select(CMECS_SC_Category)) %>%
  mutate(sub_area = as.numeric(st_area(geom))) %>%
  st_drop_geometry() %>%
  dplyr::select(c(ID, numid,sub_area,CMECS_SC_Category))%>%
  group_by(ID, numid,CMECS_SC_Category) %>%
  dplyr::summarize(sub_area= sum(sub_area)) %>%
  ungroup() %>%
  distinct()

check <- hex_substrate %>%
 group_by(CMECS_SC_Name) %>%
  dplyr::summarize(tot_area = sum(sub_area)/1e6) %>%
  ungroup() %>%
  arrange(-tot_area) %>%
  mutate(rank_hex = dense_rank(-tot_area))

check2 <- hex_substrate2 %>%
 group_by(CMECS_SC_Category) %>%
  dplyr::summarize(tot_area = sum(sub_area)/1e6) %>%
  ungroup() %>%
  arrange(-tot_area) %>%
  mutate(rank_hex = dense_rank(-tot_area))



track_substrate <- track_buffer %>%
  st_intersection(substrate %>%
                    dplyr::select(CMECS_SC_Name)) %>%
  mutate(sub_area = as.numeric(st_area(geom))) %>%
  st_drop_geometry() %>%
  dplyr::select(c(trip_date,sub_area,CMECS_SC_Name))%>%
```

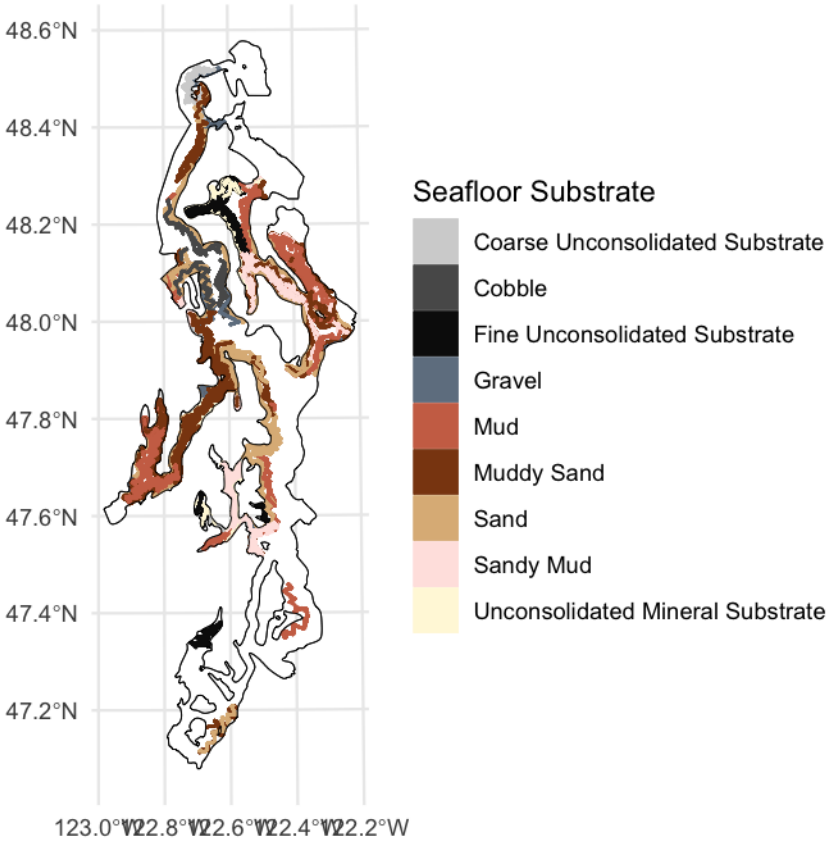
```
  group_by(trip_date,CMECS_SC_Name) %>%
  dplyr::summarize(sub_area= sum(sub_area)) %>%
  ungroup() %>%
  distinct()

check2 <- track_substrate %>%
  dplyr::select(c(trip_date,sub_area,CMECS_SC_Name))%>%
  group_by(trip_date,CMECS_SC_Name) %>%
  dplyr::summarize(sub_area= sum(sub_area)) %>%
  ungroup() %>%
  distinct() %>%
 group_by(CMECS_SC_Name) %>%
  dplyr::summarize(tot_area = sum(sub_area)/1e6) %>%
  ungroup() %>%
  arrange(-tot_area) %>%
  mutate(rank_track = dense_rank(-tot_area)) %>%
  full_join(check, by = "CMECS_SC_Name")



track_substrate <- track_buffer %>%
  st_intersection(substrate %>%
                    dplyr::select(CMECS_SC_Name)) %>%
  mutate(sub_area = as.numeric(st_area(geom))) %>%
  st_drop_geometry() %>%
  dplyr::select(c(trip_date, numid,sub_area,CMECS_SC_Name))%>%
  group_by(trip_date, numid,CMECS_SC_Name) %>%
  dplyr::summarize(sub_area= sum(sub_area)) %>%
  ungroup() %>%
  group_by(trip_date, numid) %>%
  slice_max(sub_area, n = 1, with_ties = FALSE) %>%
  ungroup()


check <- track_substrate %>%
 group_by(CMECS_SC_Name) %>%
  dplyr::summarize(tot_area = sum(sub_area)/1e6)

substrate_new_code <- substrate %>%
  mutate(new_cmecs = ifelse(startsWith(CMECS_SC_Name, "Anthro")|
                      CMECS_SC_Category == "Biogenic Substrate"|
                      CMECS_SC_Name =="Megaclast" |
                      CMECS_SC_Name =="Bedrock" |
                      CMECS_SC_Name =="Gravel Mixes" |
                      CMECS_SC_Name == "Muddy Sandy Gravel" |
                      CMECS_SC_Name == "Sandy Gravel" |
                      CMECS_SC_Name == "Muddy Gravel"|
                        CMECS_SC_Name == "Gravelly",
                      "Gravel",
                    ifelse(CMECS_SC_Name =="Silt-Clay" |
                             CMECS_SC_Name == "Clay" ,
                           "Fine Unconsolidated Substrate",
                    ifelse(CMECS_SC_Name == "Cobbley Sandy Mud"|
```

```
            CMECS_SC_Name == "Gravelly Mud" |
            CMECS_SC_Name == "Slightly Gravelly Mud", "Mud",
            ifelse(CMECS_SC_Name == "Gravelly Muddy Sand"|
                         CMECS_SC_Name == "Cobbley Sand"|
                         CMECS_SC_Name == "Slightly Cobbley Sand"|
                         CMECS_SC_Name == "Gravelly Sand" |
                         CMECS_SC_Name == "Slightly Gravelly Sand"|
                         CMECS_SC_Name == "Slightly Gravelly Muddy Sand",
                   "Sand",
                ifelse(CMECS_SC_Name == "Granule" |
                          CMECS_SC_Name == "Pebble"|
                          CMECS_SC_Name == "Boulder"|
                       CMECS_SC_Name =="Rock Substrate",
                                "Coarse Unconsolidated Substrate",
                  ifelse(CMECS_SC_Name == "Sandy Cobble","Cobble",
                        CMECS_SC_Name)))))))

check <- substrate_new_code %>%
  distinct(CMECS_SC_Name, new_cmecs)

track_substrate <- track_buffer %>%
  st_intersection(substrate_new_code %>%
                    dplyr::select(new_cmecs)) %>%
  mutate(sub_area = as.numeric(st_area(geom))) %>%
  st_drop_geometry() %>%
  dplyr::select(c(trip_date, numid,sub_area,new_cmecs))%>%
  group_by(trip_date, numid,new_cmecs) %>%
  dplyr::summarize(sub_area= sum(sub_area)) %>%
  ungroup() %>%
  group_by(trip_date, numid) %>%
  slice_max(sub_area, n = 1, with_ties = FALSE) %>%
  ungroup()

 check <- track_substrate %>%
   group_by(new_cmecs) %>%
   dplyr::summarize(tot_area = sum(sub_area)/1e6)


track_static_sub <- track_rie_bpid %>%
  full_join(track_substrate, by = c("numid","trip_date")) %>%
  dplyr::select(-sub_area)


ggplot()+
  geom_sf(data = puget, fill="gray")+
  geom_sf(data = track_static_sub, aes(fill= as.character(new_cmecs)))
```




```
hex_substrate <- hex_grids %>%
  st_intersection(substrate_new_code %>%
                    dplyr::select(new_cmecs)) %>%
  mutate(sub_area = as.numeric(st_area(geom))) %>%
  st_drop_geometry() %>%
  dplyr::select(c(ID, numid,sub_area,new_cmecs))%>%
  group_by(ID, numid,new_cmecs) %>%
  dplyr::summarize(sub_area= sum(sub_area)) %>%
  ungroup() %>%
  group_by(ID, numid) %>%
  slice_max(sub_area, n = 1, with_ties = FALSE) %>%
  ungroup()


check <- hex_substrate %>%
 group_by(new_cmecs) %>%
  dplyr::summarize(tot_area = sum(sub_area)/1e6)



hex_static_sub <- hex_rie_bpid %>%
  full_join(hex_substrate, by = c("numid","ID")) %>%
  dplyr::select(-sub_area) %>%
  mutate(new_cmecs = ifelse(is.na(new_cmecs), "Sand", new_cmecs))
```

```
ggplot()+
  geom_sf(data = puget, fill="gray")+
  geom_sf(data = hex_static_sub, aes(fill= as.character(new_cmecs)))
```



Get shore type

- `Dclass_cut.gpk` is the geopackage for the Dethier Class which was adapted from the WA ShoreZone Inventory in 2014 by . The html provides further description. It is simply the clipped version within the study in gpkg format of the downloadable file from the link above. The clipping was originally done in *QGIS*. the gpkg is used as the shoreline for clipping purposes. It is later further cut. *CRS: 26910 (meters)*

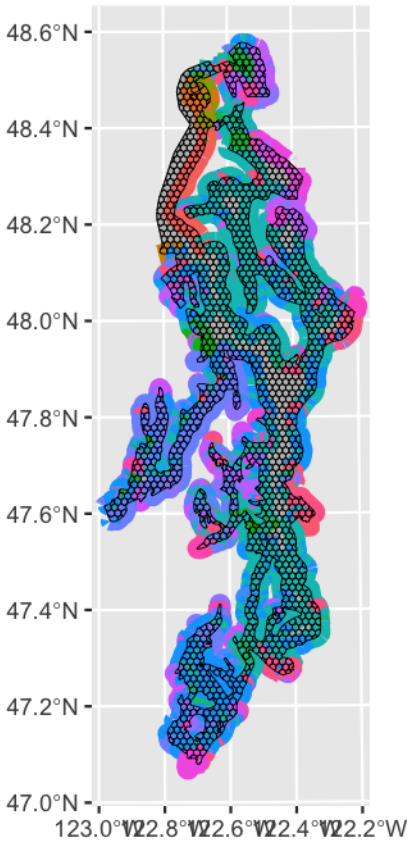
```
length(unique(shoreline$DETH_CLASS))
```

```
## [1] 54
```

```
shore_clip <- st_intersection(shoreline, studyarea)

length(unique(shore_clip$DETH_CLASS))
```

```
## [1] 41
```

```
ggplot()+
  geom_sf(data = puget, fill="gray")+
  geom_sf(data=shore_clip, aes(color =as.factor(DETH_CLASS), lwd=3)) +
  geom_sf(data = hex_grids, fill =NA, color = "black") +
  theme(legend.position = "none")
```



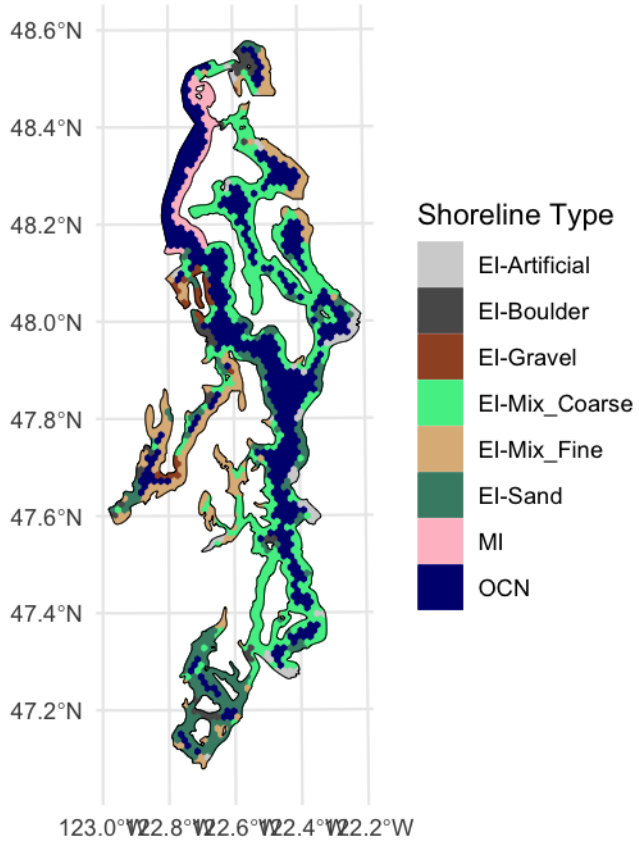

```
shore_hex <- hex_grids %>%
  st_intersection(shoreline %>%
                    dplyr::select(CLASS_NAME)) %>%
  mutate(sh_len = as.numeric(st_length(geom))) %>%
  st_drop_geometry() %>%
  dplyr::select(c(ID, numid,sh_len,CLASS_NAME))%>%
  group_by(ID, numid,CLASS_NAME) %>%
  dplyr::summarize(sh_len= sum(sh_len), .groups="drop") %>%
  ungroup() %>%
  distinct()


check <- shore_hex %>%
 group_by(CLASS_NAME) %>%
  dplyr::summarize(tot_len = sum(sh_len)/1000) %>%
  arrange(CLASS_NAME,tot_len)


check_shore <- hex_grids %>%
```

```
  st_buffer(dist=100) %>%
  st_intersection(shoreline %>%
                    dplyr::select(CLASS_NAME)) %>%
  mutate(sh_len = as.numeric(st_length(geom))) %>%
  #st_drop_geometry() %>%
  dplyr::select(c(ID, numid,sh_len,CLASS_NAME))%>%
  group_by(ID, numid,CLASS_NAME) %>%
  dplyr::summarize(sh_len= sum(sh_len), .groups="drop") %>%
  ungroup() %>%
  group_by(ID, numid) %>%
  slice_max(sh_len, n = 1, with_ties = FALSE) %>%
  ungroup()

#length(unique(check_shore$CLASS_NAME))
#unique(check_shore$CLASS_NAME)


#ggplot()+
#  geom_sf(data = puget, fill="gray")+
#  geom_sf(data=check_shore, aes(color =as.factor(CLASS_NAME), lwd=3)) +
#  geom_sf(data = hex_grids, fill =NA, color = "black") +
#  theme(legend.position = "none")

## look at prop
check_shore <- check_shore %>%
  st_drop_geometry() %>%
  group_by(CLASS_NAME) %>%
  count() %>%
  mutate(per = n/nrow(check_shore)*100) %>%
  arrange(CLASS_NAME)

###


## now with tracklines
shore_track <- track_buffer %>%
  st_buffer(dist = 300) %>%
  st_intersection(shoreline %>%
                    dplyr::select(CLASS_NAME)) %>%
  mutate(sh_len = as.numeric(st_length(geom))) %>%
  st_drop_geometry() %>%
  dplyr::select(c(trip_date,sh_len,CLASS_NAME))%>%
  group_by(trip_date,CLASS_NAME) %>%
  dplyr::summarize(sh_len= sum(sh_len), .groups="drop") %>%
  ungroup() %>%
  distinct()


check <- shore_track %>%
 group_by(CLASS_NAME) %>%
  dplyr::summarize(tot_len = sum(sh_len)/1000) %>%
  arrange(CLASS_NAME,tot_len)
```

```
check_shore2 <- track_buffer %>%
  st_buffer(dist = 300) %>%
  st_intersection(shoreline %>%
                    dplyr::select(CLASS_NAME)) %>%
  mutate(sh_len = as.numeric(st_length(geom))) %>%
  #st_drop_geometry() %>%
  dplyr::select(c(trip_date,sh_len,CLASS_NAME))%>%
  group_by(trip_date,CLASS_NAME) %>%
  dplyr::summarize(sh_len= sum(sh_len), .groups="drop") %>%
  ungroup() %>%
  group_by(trip_date) %>%
  slice_max(sh_len, n = 1, with_ties = FALSE) %>%
  ungroup()

#length(unique(check_shore2$CLASS_NAME))
#unique(check_shore2$CLASS_NAME)


#ggplot()+
#  geom_sf(data = puget, fill="gray")+
#  geom_sf(data=check_shore2, aes(color =as.factor(CLASS_NAME), lwd=3)) +
#  geom_sf(data = track_buffer, fill =NA, color = "black") +
#  theme(legend.position = "none")

## look at prop
check_shore2 <- check_shore2 %>%
  st_drop_geometry() %>%
  group_by(CLASS_NAME) %>%
  count() %>%
  mutate(per = n/nrow(check_shore2)*100) %>%
  arrange(CLASS_NAME)

shore_new_code <- shoreline %>%
  mutate(shore_code = ifelse(startsWith(CLASS_NAME,
                                        "Marine, intertidal"),
                             "MI",
                      ifelse(startsWith(CLASS_NAME,
                                "Estuarine, intertidal, art"),
                             "EI-Artificial",
                      ifelse(startsWith(CLASS_NAME,
                             "Estuarine, intertidal, boulder")|
                             startsWith(CLASS_NAME,
                              "Estuarine, intertidal, hardpan")|
                             startsWith(CLASS_NAME,
                              "Estuarine, intertidal, reef")|
                             startsWith(CLASS_NAME,
                              "Estuarine, intertidal, bedrock"),
                             "EI-Boulder",
                      ifelse(startsWith(CLASS_NAME,
                             "Estuarine, intertidal, gravel"),
                             "EI-Gravel",
                      ifelse(startsWith(CLASS_NAME,
                          "Estuarine, intertidal, mixed coarse"),
```

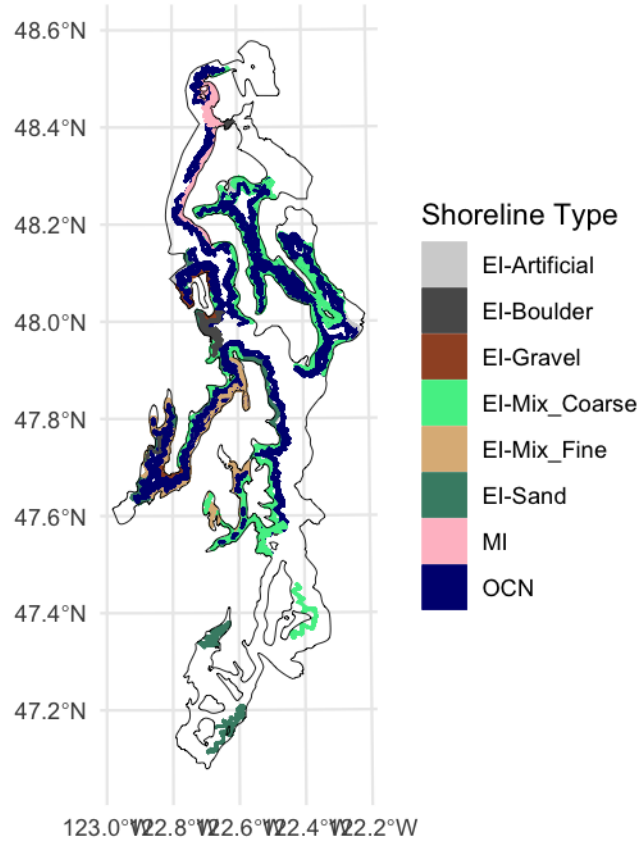
```
                        "EI-Mix_Coarse",
                      ifelse(startsWith(CLASS_NAME,
                        "Estuarine, intertidal, mixed fine") |startsWith(CLASS_NAME,
                        "Estuarine, intertidal, mud") |
                        startsWith(CLASS_NAME,
                        "Estuarine, intertidal, organic"),
                        "EI-Mix_Fine",
                      ifelse(startsWith(shore_code,
                        "Estuarine, intertidal, sand"),
                      "EI-Sand",
                      CLASS_NAME))))))))

check <- shore_new_code %>%
  distinct(CLASS_NAME,shore_code)


shore_hex <- hex_grids %>%
  st_buffer(dist=500) %>%
  st_intersection(shore_new_code %>%
                    dplyr::select(shore_code)) %>%
  mutate(sh_len = as.numeric(st_length(geom))) %>%
  st_drop_geometry() %>%
  dplyr::select(c(ID, numid,sh_len,shore_code))%>%
  group_by(ID, numid,shore_code) %>%
  dplyr::summarize(sh_len= sum(sh_len), .groups="drop") %>%
  ungroup() %>%
  group_by(ID, numid) %>%
  slice_max(sh_len, n = 1, with_ties = FALSE) %>%
  ungroup()

check <- shore_hex %>%
  group_by(shore_code) %>%
  summarise(tot = sum(sh_len)/1000)

check <- hex_static_all %>%
 filter(!ID %in% shore_hex$ID)

check <- hex_static_sub %>%
 filter(!ID %in% shore_hex$ID)




hex_static_all <- hex_static_sub %>%
  full_join(centroids_in_water %>%
              st_drop_geometry() %>%
              select(c(dshore, numid,ID)),
            by = c("numid","ID")) %>%
  full_join(shore_hex, by = c("numid","ID")) %>%
  mutate(shore_code = ifelse(is.na(shore_code),
                             "OCN", shore_code)) %>%
  dplyr::select(-sh_len)
```

```
length(unique(hex_static_all$shore_code))

ggplot()+
  geom_sf(data = puget, color="black")+
  geom_sf(data=hex_static_all, aes(fill =shore_code), color=NA)+
 # geom_sf(data= shore_new_code, aes(color=shore_code))+
  scale_fill_manual(values=c("lightgray","gray35",
                             "sienna","seagreen2","burlywood",
                             "aquamarine4","pink","navy"))+
  scale_color_manual(values=c("lightgray","gray35",
                              "sienna","seagreen2","burlywood",
                              "aquamarine4","pink","navy"))+
  theme_minimal()
```




```
shore_track <- track_buffer %>%
  st_buffer(dist = 300) %>%
  st_intersection(shore_new_code %>%
                    dplyr::select(shore_code)) %>%
  mutate(sh_len = as.numeric(st_length(geom))) %>%
  st_drop_geometry() %>%
  dplyr::select(c(trip_date, numid,sh_len,shore_code))%>%
  group_by(trip_date, numid,shore_code) %>%
  dplyr::summarize(sh_len= sum(sh_len), .groups="drop") %>%
  ungroup() %>%
  group_by(trip_date, numid) %>%
```

```
  slice_max(sh_len, n = 1, with_ties = FALSE) %>%
  ungroup()


check <- shore_track %>%
  group_by(shore_code) %>%
  summarise(tot = sum(sh_len)/1000)

check <- track_static_sub %>%
 filter(!trip_date %in% shore_track$trip_date)



ggplot()+
  geom_sf(data= check)+
  geom_sf(data = shore_new_code, aes(color =shore_code))



track_static_all <- track_static_sub %>%
  full_join(shore_track, by = c("numid","trip_date")) %>%
  mutate(shore_code = ifelse(is.na(shore_code),
                             "OCN", shore_code)) %>%
  dplyr::select(-sh_len)

length(unique(track_static_all$shore_code))

ggplot()+
  geom_sf(data = puget, color="black")+
  geom_sf(data=track_static_all, aes(fill =shore_code), color=NA)+
 # geom_sf(data= shore_new_code, aes(color=shore_code))+
  scale_fill_manual(values=c("lightgray","gray35",
                             "sienna","seagreen2","burlywood",
                             "aquamarine4","pink","navy"))+
  scale_color_manual(values=c("lightgray","gray35",
                              "sienna","seagreen2","burlywood",
                              "aquamarine4","pink","navy"))+
  theme_minimal()

table(track_static_all$shore_code)
table(track_static_all$new_cmecs)
```



Save

```
track_static_all_df <- track_static_all %>%
  st_drop_geometry()

hex_static_all_df <- hex_static_all %>%
  st_drop_geometry()

st_write(hex_static_all, "GIS_COVS/all_static_hex.gpkg",delete_layer = TRUE)
st_write(track_static_all, "GIS_COVS/all_static_track.gpkg",delete_layer = TRUE)

write.csv(track_static_all_df, "GIS_COVS/all_static_track.csv")
write.csv(hex_static_all_df, "GIS_COVS/all_static_hex.csv")
```